

Al-Rahmeh Association for Animals

جمعية الرحمة للرفق بالحيوان

A rescue website rebuilt as one Flask process over one SQLite file, built over eight working days as a series of vertical slices, each merged by pull request.

Repository github.com/saImansour17/alrahmeh-animals Stack: Flask 3.1 · sqlite3 · React (Vite) 4 October 2026

102

COMMITTS

23

MERGED PRS

3

DOMAINS

405

TESTS

100%

LINE + BRANCH COV.

5

ADRS

1 The problem and who it serves

Al-Rahmeh is a small animal rescue founded in Madaba, Jordan, in January 2018. Its existing website, audited for another course, failed its users in three concrete ways. Each failure became a requirement.

Hard-coded impact numbers 01

~~"More than 300 pets found homes" was typed into the HTML.~~
Every homepage figure is computed from records when the page loads.

A buried donation flow 02

~~Donating meant leaving the site for a GoFundMe page.~~
Donate sits in every header: card, CliQ and bank transfer in one place.

Forms with no backend 03

~~"Adopt Now" and "Foster Now" opened Google Forms.~~
A request is a record tied to a real animal; a staff decision moves it through a guarded lifecycle.

Stakeholders

Staff: 2 to 3 people who admit animals, decide requests, record gifts

Adopters and fosterers

Donors in Jordan and abroad

Volunteers

2 SDLC model and goals

Model: **iterative and incremental, one vertical slice per day**. Each increment was a complete slice (model, repository, service, routes, tests, documentation) on its own feature branch, merged by pull request. Planning was Kanban-like: a written schedule in `CLAUDE.md` set each day's slice, and the next day's plan was revised at the end of every session.

MODEL	HOW IT FITS A SOLO, TEN-DAY PROJECT WITH MOVING REQUIREMENTS	VERDICT
Waterfall	Needs requirements fixed up front. Mine changed four times in seven days (Section 2.2).	REJECTED
Scrum	Sprint planning, reviews and retrospectives are team ceremonies; for one person they cost more than they return.	REJECTED
Iterative + incremental	Every day ends with something that runs. Branch-per-feature also meets the brief's rules: no day above 40% of commits, decisions logged as they happen.	CHOSEN

SMART GOALS AND RESULTS

GOAL (SPECIFIC, MEASURABLE, TIME-BOUND)	RESULT
Replace all three audit failures with working features by 2026-10-04.	MET done on 2026-10-03
≥70% line coverage of business logic in every domain by the end of the second domain (2026-09-30).	MET 100% on 09-30, kept since (405 tests)
≥12 commits over ≥6 days, no day above 40%, pushed.	MET 102 commits, 8 days, busiest 20/102 (19.6%)
Five ADRs across ≥3 commit dates.	MET 4 dates: Sep 28, 29, 30, Oct 1
Clean clone, <code>pip install</code> , <code>python app.py</code> serves the whole site in under 5 s with no Node, no <code>.env</code> , no manual step.	MET ready in 1.8 s on 2026-10-04; details in README

2.2 The lifecycle in practice

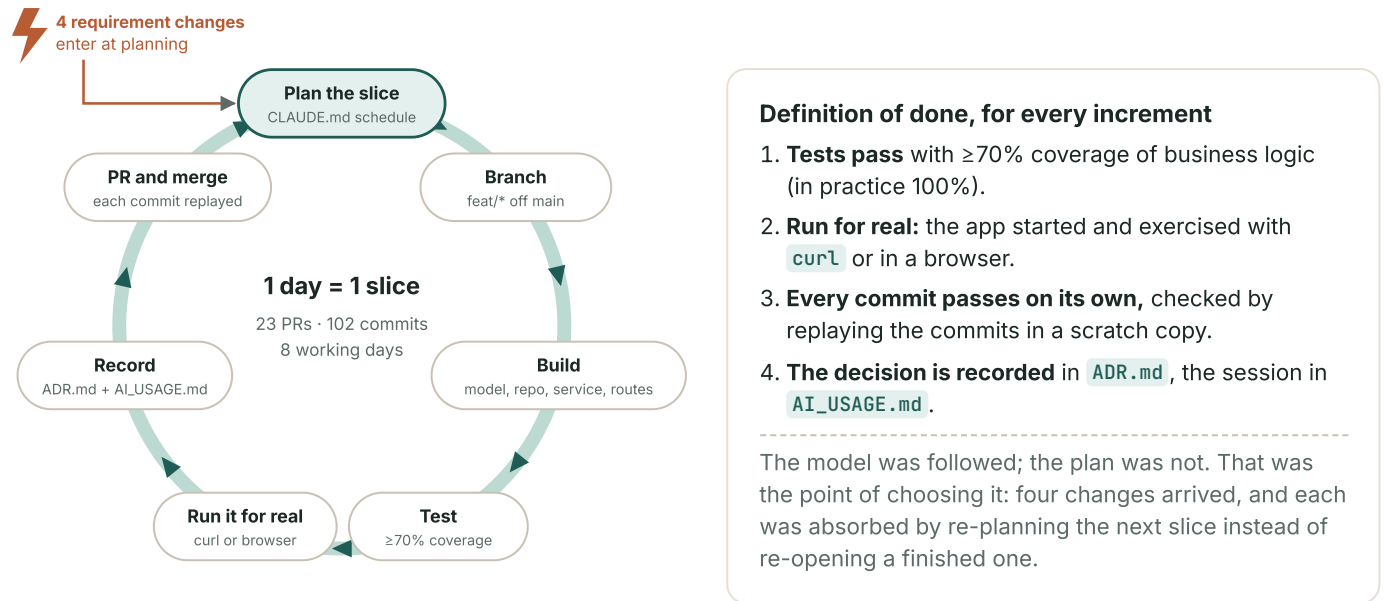


Figure 1. The daily increment. Requirement changes only ever enter at planning, so a pivot never lands half-way through a slice.

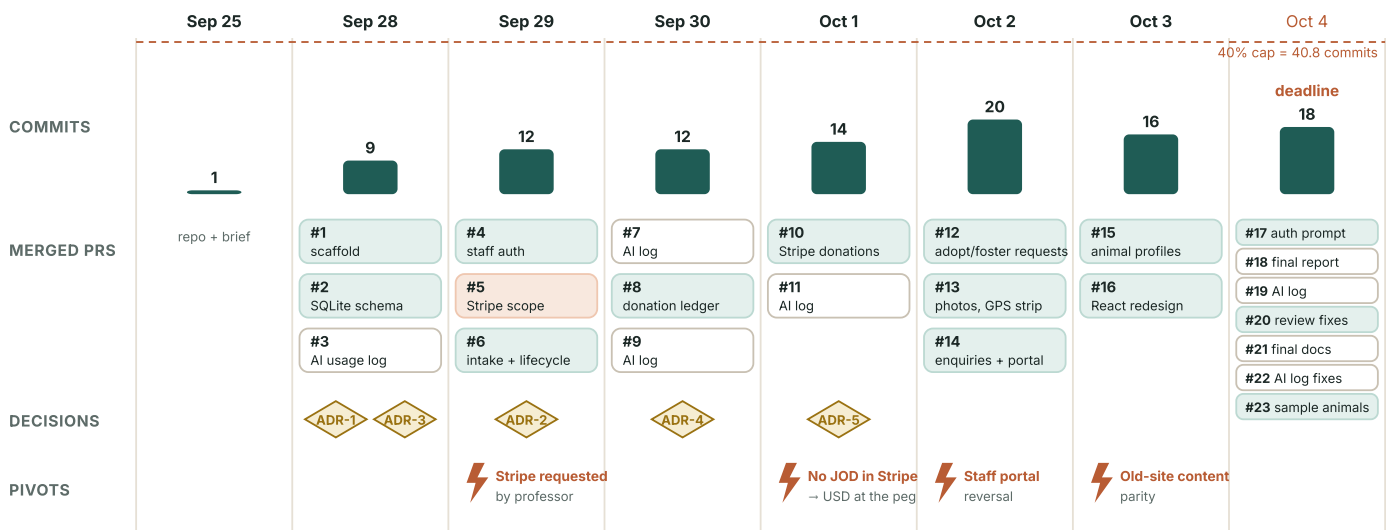


Figure 2. What actually shipped, from `git log`. Bars are commits per day (none comes near the 40% cap); chips are merged PRs (■ feature ■ docs ■ scope change); diamonds are ADR commits.

PLANNED	WHAT HAPPENED, AND WHY
Stripe out of scope (old ADR-5).	Brought in on Sep 29 at my professor's request to demonstrate Adapter and Dependency Inversion (PR #5). ADR-5 moved to the decision <i>not</i> to build a Jordanian production gateway.
Card payments in JOD.	My Stripe test account cannot charge JOD. Cards are charged in USD at the Central Bank of Jordan peg (709 fils per dollar) and converted back inside the adapter, so the ledger stays in fils (PR #10).
No staff pages.	Reversed on Oct 2: in-person adoptions and cash gifts never touch the website, so the totals can only be true if staff can record them. A staff portal and an offline-adoptions record were added (PRs #14, #16).
A thin frontend, last.	On review the new site had <i>less</i> than the old one. All old content was carried over, a third domain stores contact and volunteer messages, and the design was rebuilt from three reference designs (PRs #14 to #16).

Unplanned additions from security review: a shared staff password (PR #4), GPS stripping on uploaded photos (PR #13), append-only triggers on the donation ledger (PR #8), no browser login prompt on a refused staff request, so the browser never stores the password (PR #17), and, from an end-to-end review, a manual move back to available now ends the approved adoption and oversized ids answer 404 instead of 500 (PR #20).

3 Architecture

One Flask process serves both the JSON API and the compiled React app, and stores everything in one SQLite file plus a photos folder under `DATA_DIR`. `create_app()` is the **composition root**: the only place that chooses concrete classes, and the only module that imports more than one domain.

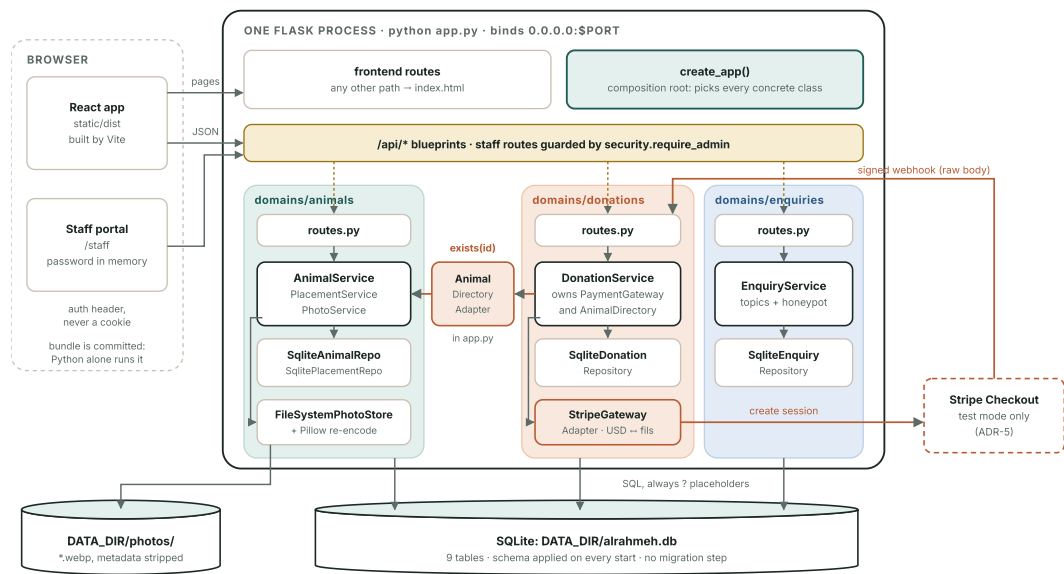


Figure 3. Architecture as built. Clay arrows mark the only two translations: Stripe (`StripeGateway`) and the other domain (`AnimalDirectoryAdapter`).

The seam. The three domains never import one another and share only primary-key values; `tests/test_domain_boundary.py` fails if that changes (ADR-2). In each domain, `models.py` validates, `repository.py` alone runs SQL (always ? placeholders), `service.py` holds the rules and owns its interfaces, and `routes.py` only translates HTTP.

3.1 Design patterns, by category

My rule: **claim only what is really in the code; never add a pattern to fill a category**. A forced pattern is worse code I would still have to defend.

CATEGORY	IN THE CODE	LOOKS LIKE A PATTERN, NOT CLAIMED, AND WHY
Creational	Application factory: <code>create_app()</code> builds every object once and injects it (Dependency Injection); <code>_payment_gateway()</code> picks the gateway, or none, from config; <code>from_payload</code> constructors validate while building.	Factory Method: no subclass decides what to build. Singleton: <code>Config</code> and <code>Database</code> are passed in, never global, so each test builds its own. Builder: frozen dataclasses are built in one step.
Structural	Adapter, twice: <code>StripeGateway</code> (Stripe SDK to <code>PaymentGateway</code> , cents to fils, Stripe errors to ours) and <code>AnimalDirectoryAdapter</code> (<code>AnimalService</code> to <code>AnimalDirectory</code>). Repository in every domain.	Decorator: <code>@require_admin</code> is the idea applied to a function, not an object. Adapter for <code>FileSystemPhotoStore</code> : nothing incompatible is adapted; it is DIP. Proxy: <code>StaleRead</code> exists only in tests.
Behavioural	None claimed. The rules are data: <code>ALLOWED_TRANSITIONS</code> and <code>ACCEPTS_REQUESTS</code> are lookup tables, tested over every pair.	State: statuses differ only in allowed moves, so a table beats four classes. Strategy: the gateway is chosen once at startup, never swapped. Command / Observer: no in-process events. Null Object: no gateway is <code>None</code> , answered with 503.

3.2 SOLID: where the code had to change

PRINCIPLE	WHAT CHANGED TO ENFORCE IT, AND WHERE TO SEE IT
S Single responsibility	Adoption and foster requests got their own <code>PlacementService</code> instead of growing <code>AnimalService</code> into a god class (Oct 2, PR #12). Only <code>payments.py</code> imports <code>stripe</code> (a test enforces it); only <code>db/connection.py</code> imports <code>sqlite3</code> .
O Open/closed	Lifecycle and request rules became tables, not <code>if/elif</code> chains; a new payment provider is one class plus one line in <code>create_app</code> . A new status needs no service code.
L Liskov substitution	The fake gateway moved out of production code into the tests (Oct 1); every fake returns the same dataclasses and raises the same errors as the real class. <code>StaleRead</code> runs unchanged against the service.
I Interface segregation	The ledger sees only <code>AnimalDirectory.exists()</code> , never the whole <code>AnimalService</code> . <code>PaymentGateway</code> has two methods; each repository Protocol lists only what its service calls.
D Dependency inversion	Stripe came in (PR #5, my professor's request) behind a <code>PaymentGateway</code> Protocol the service owns; commits were reordered so the adapter imports the service's types, never the reverse. The Amman clock (Sep 30) and <code>PhotoStore</code> (Oct 2) are injected too. Concrete classes appear only in <code>create_app()</code> .

4 Database model

The schema applies itself on every start with `CREATE TABLE IF NOT EXISTS`, so a fresh container and an existing one take the same path and there is no migration step. Every table added after day one is a **new table, never a new column**, because that statement never alters a table that already exists (ADR-3).

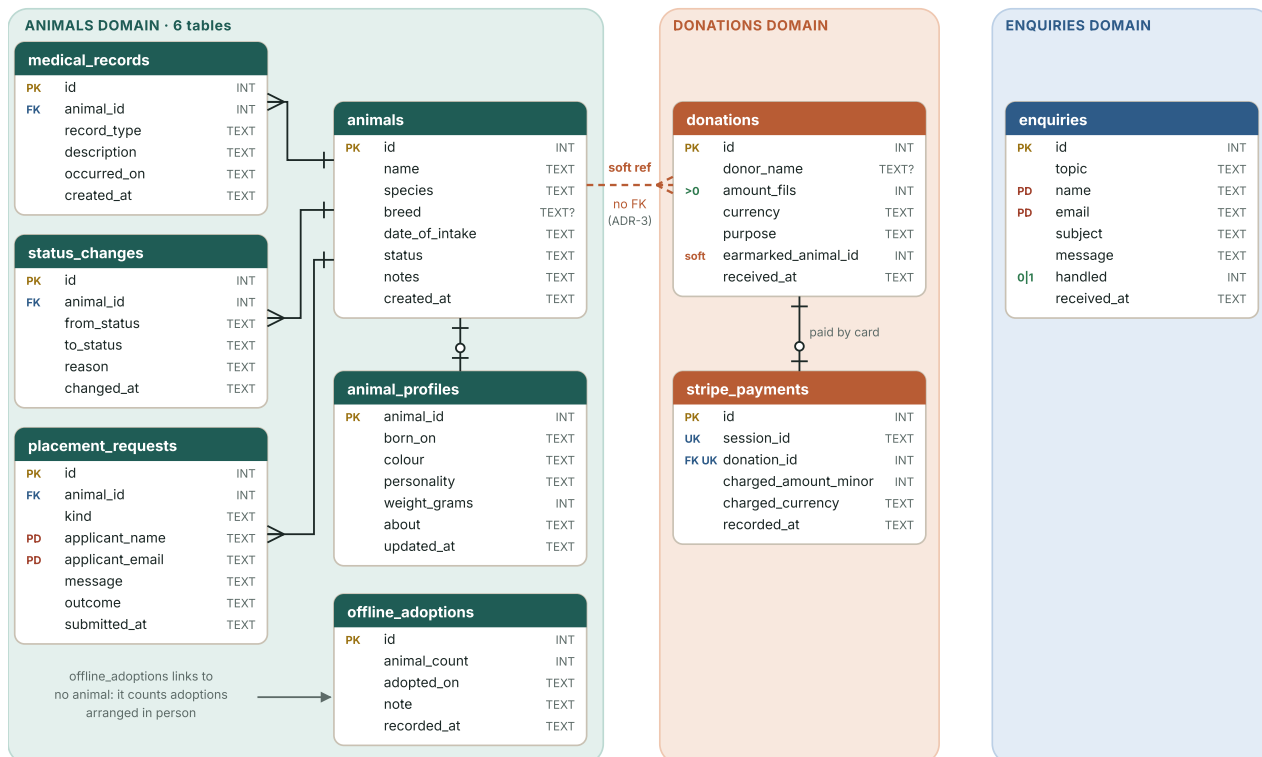


Figure 4. All nine tables. **PK** primary key · **FK / UK** foreign or unique key · **PD** personal data, staff-only · **soft** id with no foreign key · solid lines are real foreign keys with `ON DELETE CASCADE`; the dashed line is the only link across domains.

Money is integer fils. 1 JOD = 1000 fils. Homepage totals are a SUM, and binary floats drift. Amounts arrive as strings and are checked character by character.

The ledger is append-only, enforced by SQLite. The repository has no update or delete, and two triggers refuse UPDATE and DELETE on `donations`.

The cross-domain link is soft. `earmarked_animal_id` has no FK, so the ledger can move to its own service without a data migration; the service checks the id instead.

A card payment is recorded once. `session_id` is UNIQUE and both rows are written in one transaction, so a replayed webhook rolls back.

"Animals found homes" is adopted animals plus `offline_adoptions`: every part of the figure is a record someone entered.

Weight is whole grams, age is a birth date, so no decimal reaches the database and an age is never out of date.

5 Testing

405
TESTS

100%
LINE + BRANCH, LOGIC

98%
ALL APP CODE

16/16
STATUS PAIRS

8/8
REQUEST × STATUS

100% line and branch coverage of the business logic (the 11 model, repository and service files in `domains/`; route handlers are excluded in `.coveragerc` as framework glue), and **98% across all application code**. Several tests were confirmed to fail when the bug they guard is put back.

Measured with `pytest --cov=domains --cov-report=term-missing` (1077 of 1077 lines; 188 of 188 branches with `--cov-branch`). Services are tested against a **real SQLite database** in a temporary folder rather than mocks, so the SQL is exercised too, and **small input spaces are enumerated**: every status pair, every request-kind and status combination, every malformed amount I could find (Arabic-Indic digits, a trailing newline). Fakes exist only at declared seams (`FakeAnimalDirectory`, `FakeGateway`, an in-memory photo store). Stripe webhooks are signed locally with a dummy secret, so no network is needed. For example, swapping `[0-9]` for `\d` fails the Arabic-digit test. The tests also caught two of my own bugs: a decompression-bomb guard that was never switched on, and Windows file-time rounding on replaced photos.

Left thin, deliberately: route handlers are excluded from coverage (glue); the race guard uses a simulated stale read, not threads; no frontend unit tests (checked in a real browser); Stripe's live API is exercised by hand with the Stripe CLI.

6 Decisions I made, and why

The five ADRs are in `ADR.md`; the rest were made in the middle of a slice (pattern choices are in Section 3.1). Each row says what I chose, the reason that settled it, and what I gave up.

DECISION	WHY	REJECTED / COST
ADR-1 Flask 3.1 + stdlib <code>sqlite3</code> , no ORM	Five tables, no async work, and I must explain every line cold.	Django: ORM, admin and migrations unused. FastAPI: nothing to <code>await</code> . Cost: hand-written SQL.
ADR-2 Domains never import each other	A later split becomes “move a folder, swap one adapter”, proven by a test.	A layered layout puts the seam through every file. Cost: duplicated error handlers.
ADR-3 Soft cross-domain id; money as integer fils	No FK crosses the seam; a SUM of integers is exact.	Real FK (needs a data migration to split); <code>REAL</code> amounts (drift). Cost: the service validates the id.
ADR-4 Test rules on real SQLite, enumerate small spaces	A mock only repeats my assumptions about the SQL, e.g. empty SUM = NULL.	Mock-heavy tests; counting routes toward 70%. Cost: concurrency shown by simulation only.
ADR-5 No production Jordanian gateway; Stripe as a test adapter	Stripe does not onboard Jordanian businesses; PayTabs/HyperPay need the rescue’s documents.	An untestable PayTabs adapter; Stripe calls inside the service. Cost: test mode only.
Charge USD at the 0.709 peg, store fils	The test account has no JOD; the ledger must stay single-currency.	Recording each gift in its charged currency would break ADR-3.
Reverse “no staff pages”	Cash gifts and in-person adoptions would make the live totals false.	Kept the old plan: honest code, dishonest numbers.
Staff password in memory, sent as a header	No cookie means no cross-site request forgery to defend against.	Session cookies + CSRF tokens. Cost: re-enter on reload.
Commit the compiled React bundle	The deploy contract runs <code>python app.py</code> only; Node must never be needed.	Build at startup. Cost: a second manifest (approved deviation).

7 Security, omissions and next steps

Security and privacy

- **Staff access:** one `ADMIN_PASSWORD`, compared in constant time; unset means 503, never open. Exactly four public write endpoints: checkout, the signed Stripe webhook, the request form and the contact form.
- **Payments never trust the browser:** the ledger records the amount Stripe verified; return URLs come from `PUBLIC_BASE_URL`, not the Host header; a live key switches card payments off.
- **Personal data** is staff-only, never logged, never echoed in errors (tested). Photos are re-encoded to WebP, stripping GPS that could reveal a foster home.
- **Bots** meet a honeypot answered exactly like a real submission.

Deliberately not built (yet)

- **A Jordanian production gateway** (ADR-5): next is a PayTabs or HyperPay adapter behind the same interface.
- **Per-staff accounts:** no per-person audit trail; fine for three staff, first to add if the team grows.
- **Ledger corrections:** `amount_files > 0` rules out a negative compensating row.
- **Rate limiting:** deferred. In one process an in-memory limiter would work; the honeypot stops simple bots for now.
- Data retention, a full Arabic RTL version, shop checkout.

Approved deviations: two root manifests (`requirements.txt` and `package.json`) and 65 files against a guide of 50.

8 AI disclosure

*I acknowledge the use of **Claude Code (Anthropic’s Claude Opus models)** to plan the schedule and branches, propose and draft code, tests and documentation under a written brief (`CLAUDE.md`), review my decisions against the assignment, and run and verify the application. The prompts used include requests to build each feature branch against stated constraints, to list conflicts with the brief before coding, to check my own explanations against the code, and to restyle the site after reference designs I supplied; each is recorded word for word in `AI_USAGE.md`. The output of these prompts was used as a draft that I reviewed, questioned and often changed: every decision in `ADR.md` was mine, several AI recommendations were rejected or reversed (logged in `AI_USAGE.md`), and the “in my own words” column was written by me alone.*